



You make **possible**



Coding 1002: Getting Started with Python

Matthew DeNapoli – Developer Advocate
@theDeNap

DEVNET-1893.a

CISCO *Live!*

Barcelona | January 27-31, 2020



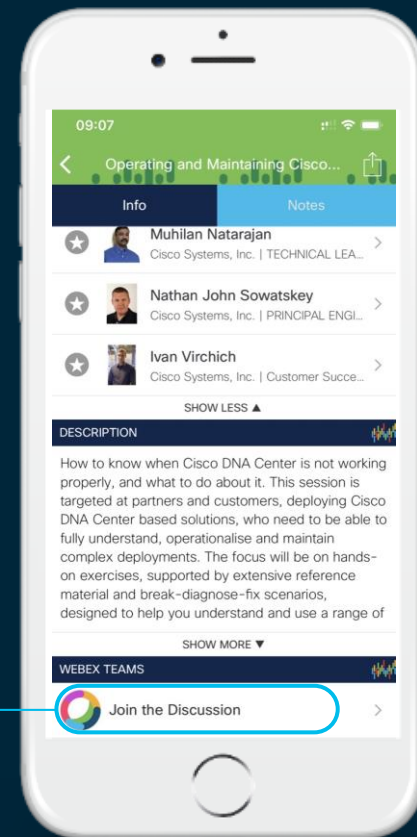
Cisco Webex Teams

Questions?

Use Cisco Webex Teams to chat with the speaker after the session

How

- 1 Find this session in the Cisco Events Mobile App
- 2 Click “Join the Discussion”
- 3 Install Webex Teams or go directly to the team space
- 4 Enter messages/questions in the team space



Agenda

- Why Python?
- Python Basics
- Python Conditionals and Loops
- Python Scripts and Execution

Why Python?

Python is...

- An interpreted language
Do not need to compile the code
- Easy to understand
Code reads like English without unnecessary syntax
- Easy to code
Do powerful things with less code
- Simple
Scripts are UTF-8 text files that can be edited in any text editor

Python...

- Maintains two stable versions
 - Python 2.x
 - Python 3.x (not backwards compatible)
- Includes pip
 - Installs packages and their dependencies
- Has a Python Interactive Shell
 - Useful for testing out code or libraries
- Has virtual environments
 - An isolated environment for installing and working with Python Packages

Why Python?

- Power and Flexibility
Create & Work With: Shell Scripts, Back-end Web APIs, Front-end UIs, Databases, Machine Learning, etc.
- Platform Flexibility
Run Your Code: Laptop, Server, VM, Container, Cloud, Cisco IOS Device
- Domain Applicability
Established online community with open source and code sharing

Python Basics

Basic Data Types

Python type()	Values (examples)
int	-128, 0, 42
float	-1.12, 0, 3.14159
bool	True, False
str	"Hello World"
bytes	b"Hello \xf0\x9f\x98\x8e"

```
>>> type(3)
<class 'int'>
>>>
>>> type(1.4)
<class 'float'>
>>>
>>> type(True)
<class 'bool'>
>>>
>>> type("Hello")
<class 'str'>
>>>
>>> type(b"Hello")
<class 'bytes'>
```

Strings

- Are any Unicode text (Python 3.x)
- Can use ' ', " ", ''' ''' and """ """
- Combined single and double quotes to:
 - include quotes in a string
 - use apostrophes in a string
- Use triple quotes for multiline strings

```
>>> single_quote = 'This is my string.'  
>>> double_quote = "This is my string."  
>>> nested_quote = 'He said, "I love Python".'  
>>> triple_quote = '''I want my string  
to be on  
separate lines.'''
```

Working with Strings

String Operations

Concatenation: **+**

Multiplication: *****

Some Useful String Methods

Composition: **"{}".format()**

Splitting: **"".split()**

Joining: **"".join()**

Length: **len("")**

```
>>> "One" + "Two"
'OneTwo'
>>>
>>> "Abc" * 3
'AbcAbcAbc'
>>>
>>> "Hi, my name is {}".format("Chris")
'Hi, my name is Chris!'
>>>
>>> "Python is cool".split(" ")
['Python', 'is', 'cool']
>>>
>>> ",".join(['Bob', 'Sue', 'Joe'])
'Bob,Sue,Joe'
>>>
>>> len("a b c")
5
```

Variables

- Reserved memory to store values
- Every variable is an object
- Created with the = assignment operator
- Are type agnostic and can change type throughout its lifetime
- Variable Names
 - Cannot start with a number [0-9]
 - Cannot conflict with a language keyword
 - Can contain: [A-Za-z0-9_ -]

```
>>> b = 7
>>> c = 3
>>> a = b + c
>>> a
10
>>>
>>> b = "Foo"
>>> c = "Bar"
>>> a = b + c
>>> a
'FooBar'
```

Variable Scope

Type	Where are they defined?	What is the scope?	Notes
global/ module variable	• In the main body of the Python script, outside of any function or class • In a function when the global keyword is used	• Throughout the whole module/file • Any file that imports the Python file • In every function	• Should be avoided when unnecessary • Global variables that are in all caps are often used to represent constants
local variable	• In a function • In a class	• In the function or class where it was defined	• When in a function, if a global and local variable has the same name, the local variable will be used • Function arguments are local variables

Variable Scope Example

```
>>> myVariable = "This is a global variable"
>>>
>>> def myFunction():
...     myLocalVariable = "This is a local variable"
...     myVariable = "This is a local variable with the same name as the global variable"
...     print(myLocalVariable)
...     print(myVariable)
...
>>>
>>> myFunction()
This is a local variable
This is a local variable with the same name as the global variable
>>>
>>> myVariable
This is a global variable
>>>
>>> myLocalVariable
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'myLocalVariable' is not defined
```

In Python, Everything is an Object!

- Objects are an encapsulation of variables and functions into a single entity
- Use the `.` (*dot*) syntax to access methods inside an object.
- View methods with `dir(obj)`
- Terminology

When contained inside an object, we call...

Variable → Attribute

Function → Method

```
>>> a = 57
>>> a.bit_length()
6
>>>
>>> b = 3.5
>>> b.is_integer()
False
>>>
>>> "WhO wRoTe THIs?".lower()
'who wrote this?'
>>>
>>> dir(a)
['_abs_', '__add__', '__and__', '__bool__', '__ceil__', '__class__',
 '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__',
 '__float__', '__floor__', '__floordiv__',
```

Advanced Data Types (Collections)

Name type()	Notes	Example
list	• Ordered list of items • Items can be different data types • Can contain duplicate items • Mutable (can be changed after created)	<code>['a', 1, 18.2]</code>
tuple	• Just like a list; except: • Immutable (cannot be changed)	<code>('a', 1, 18.2)</code>
dict (dictionary)	• Unordered key-value pairs • Keys are unique; must be immutable • Keys don't have to be the same data type • Values may be any data type	<code>{"apples": 5, "pears": 2, "oranges": 9}</code>

Working with Collections

Name type()	Creating	Accessing Indexing	Updating	Useful methods
list	l = ['a', 1, 18.2] l2 = [53, 1, 67] l3 = [[1, 2], ['a', 'b']]	>>> l[2] 18.2	>>> l[2] = 20.4 >>> l ['a', 1, 20.4]	>>> concat_l = l2 + l3 >>> concat_l [53, 1, 67, [1, 2], ['a', 'b']] >>> l2.sort() >>> l2 [1, 53, 67]
tuple	t = ('a', 1, 18.2) t2 = (1, 3, 4)	>>> t[0] 'a'	You cannot update tuples after they have been created.	>>> concat_t = t + t2 >>> concat_t ('a', 1, 18.2, 1, 3, 4)
dict	d = {"apples": 5, "pears": 2, "oranges": 9} d2 = {1: 15, 5: 'grapes', 9: [1, 2, 3]}	>>> d["oranges"] 9 >>> d.get("pears") 2	>>> d["pears"] = 6 >>> d {'apples': 5, 'pears': 2, 'oranges': 9}	>>> d.items() dict_items([('apples', 5), ('pears', 2), ('oranges', 9)]) >>> d.keys() dict_keys(['apples', 'pears', 'oranges']) >>> d.values() dict_values([5, 2, 9])

Python Conditionals and Loops

Conditional Expressions

Syntax:

operand operator operand

Ex: *x < 10*

- ✓ All expressions have minimum of one operand
- ✓ Operands are the objects that are manipulated
 - Can be a variable
 - Can be a value
- ✓ Evaluated to a Boolean

Comparison Operators:

Less than	<
Greater than	>
Less than or equal to	<=
Greater than or equal to	>=
Equal	==
Not Equal	!=
Contains element	in

Logical Operators

Syntax:

expr1 operator expr2

Ex: *x<10 and x>0*

- ✓ Logical operators join together expressions
- ✓ Evaluated to a Boolean
- ✓ Expressions are evaluated from left to right
- ✓ Can combine multiple logical operators

Logical Operators:

Logical Operator	expr1	expr2	Result
or	True	False	True
	False	True	True
	True	True	True
	False	False	False
and	True	False	False
	False	True	False
	True	True	True
	False	False	False
not	-----	True	False
	-----	False	True

Conditionals

Syntax:

```
if expression1:  
    statements1...  
elif expression2:  
    statements2...  
else:  
    statements3...
```

- ✓ Indentation is important!
- ✓ 4 spaces indent recommended
- ✓ You can nest if statements
- ✓ **elif** and **else** statements are optional

```
>>> b = 5  
>>> if b < 0:  
...     print("b is less than zero")  
... elif b == 0:  
...     print("b is exactly zero")  
... elif b > 2 and b < 7:  
...     print("b is between three and six")  
... else:  
...     print("b is something else")  
...  
b is between three and six
```

Loops | For

Syntax:

for *individual_item* **in** *iterator*:
 statements...

- ✓ Loops are used when you want to do something many times.
- ✓ Iterate over a sequence or collection
- ✓ **Iterator** can be a list, tuple, dictionary, set, string, etc.

```
>>> names = ["Chris", "Dave", "Jay"]
>>> for name in names:
...     print(name)
...
Chris
Dave
Jay
```

Loops | While

Syntax:

while *logical_expression*:
 statements...

- ✓ Executes until the **logical_expression** is false
- ✓ Watch out for infinite loops
- ✓ Use **break** to exit the loop
- ✓ Use **continue** to skip the current block

```
>>> i = 0
>>> while i < 4:
...     print(i)
...     i += 1
...
0
1
2
3
>>>
>>> while True:
...     print("In an infinite loop.")
...
In an infinite loop.
In an infinite loop.
In an infinite loop.
In an infinite loop.
...
```

Functions | Don't Repeat Yourself

Syntax:

```
def func_name (optional_args):  
    statements...  
    return value
```

- ✓ Modularize your code
 - Defining your own Functions
 - (optional) Receive arguments
 - (optional) Return a value
- ✓ **optional_args** are local variables

```
>>> def circumference(radius):  
...     result = 2 * math.pi * radius  
...     return result  
...  
>>> circumference(2)  
12.566370614359172  
  
>>>  
>>> def say_hello():  
...     print("Hello!")  
...  
>>> say_hello()  
Hello!
```

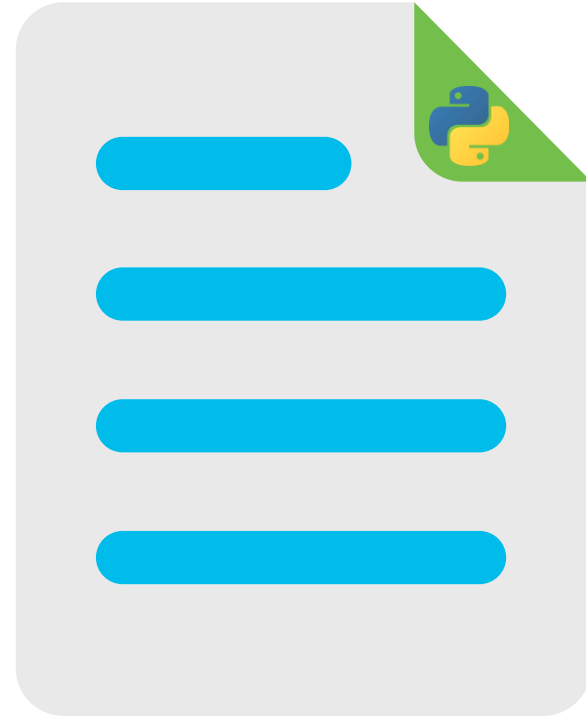
Python Scripts and Execution

Python Scripts

- ✓ File extension: *.py
- ✓ Text files (UTF-8)
- ✓ Use any text editor

Using a Python-aware editor will make your life easier

- ✓ No need to compile
- ✓ Executed with a Python Interpreter



Importing and Using Packages & Modules

Syntax:

import module

from module **import** func/const

- ✓ Import code into your script
 - Another module you wrote
 - Modules provided by companies
 - From other developers
- ✓ Can import the whole module or a specific function or constant

```
>>> import requests
>>> response = requests.get('https://google.com')
>>> response.status_code
200
>>>
>>> from requests import get
>>> get('https://google.com')
>>> response.status_code
<Response [200]>
```

Basic I/O

Get Input with `input()`

- Pass it a prompt string
- Returns the user's input as a string
- Need to convert the returned string to the correct data type

```
>>> print('a', 'b', 'c')
a b c
>>>
>>> i = input("Enter a Number: ")
Enter a Number: 1
>>> int(i)
1
```

Display Output with `print()`

- Can pass multiple values
- It will concatenate those values with separators in between (default = spaces)
- Adds a newline ('\n') to the end

Python Script Example

```
#!/usr/bin/env python

# Imports
import random

# Module Constants and Global/Module Variables
FORTUNES = [
    "There is a good chance your code will work, eventually.",
    "I see Network DevOps in your future."
]

# Module Functions and Classes
def generate_fortune() -> str:
    return random.choice(FORTUNES)

def main():
    print(generate_fortune())

# Check to see if this file is the "__main__" script being executed
if __name__ == '__main__':
    main()
```

Executing Python Scripts

Syntax:

python file_name.py

python3 file_name.py

- ✓ Use the keyword **python** or **python3** to execute the script
- ✓ Scripts are executed from the Terminal
- ✓ The Python Interpreter starts from the first line and executes each statement in succession

```
(virtual_env)$ python fortune.py
I see Network DevOps in your future.
(virtual_env)$
(virtual_env)$ python fortune.py
I see Network DevOps in your future.
(virtual_env)$ deactivate
$
$ python3 fortune.py
$ There is a good chance your code will work, eventually.
```

Learn more about the new DevNet Certifications and how you can prepare now!

Associate Level

Specialist Level

Professional Level

Expert Level

Engineering



Software



Future
Offering

Start Here | Upcoming Cisco DevNet Certifications

- Start at [Meet DevNet](#)

DEVNET-2864: Getting ready for Cisco DevNet Certifications

Offered daily at 9am, 1pm & 4pm at Meet DevNet

- Attend a [brownbag session](#)

DEVNET-4099: DevNet Certifications: Bringing software practices & software skills to networking

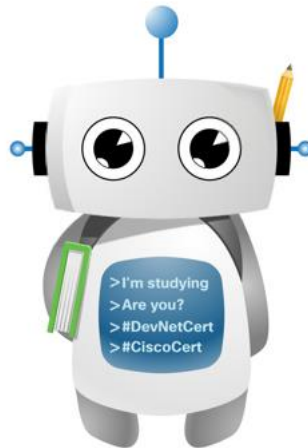
Offered daily 12:15-12:45 in the DevNet Zone Theater

- Visit the [Learning@Cisco](#) booth
- Scan this code to [sign up](#) for the latest updates or go to

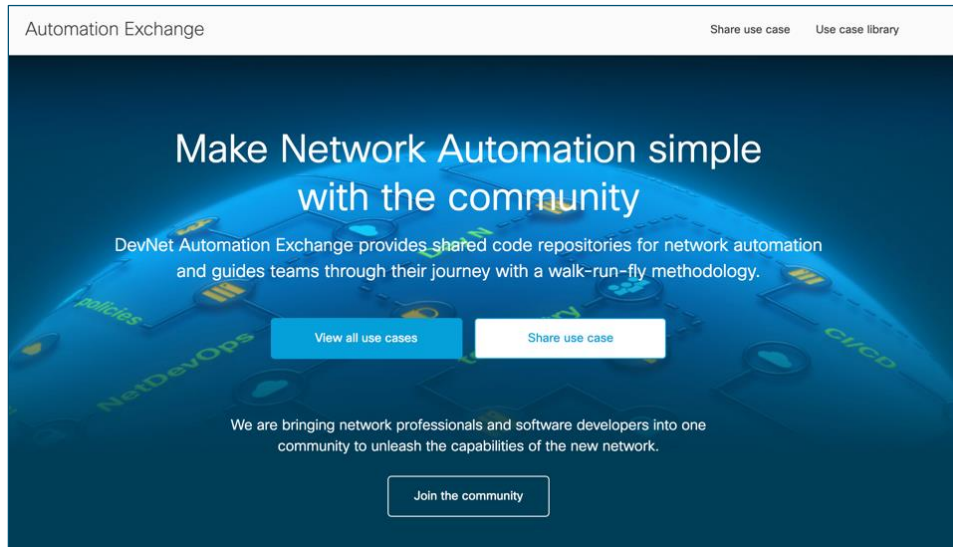


cisco *Live!*

[http://](#)

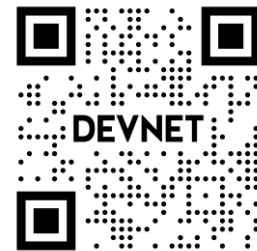


Find shared code repositories of use cases for network automation & more



Don't miss our 5
Automate Infrastructure
demos in the
DevNet Zone!

Scan this code or go to
the URL to **learn more**



<http://cs.co/20eur01>

Start at **Meet DevNet**

DEVNET-3010 [a-j] Learn how to make Network Automation Simple with the Community

Offered Monday 2pm & 5pm, Tuesday & Wednesday 10am, 2pm & 5pm, and Thursday 10am & 5pm at Meet DevNet

cisco *Live!*

Complete your online session survey



- Please complete your session survey after each session. Your feedback is very important.
- Complete a minimum of 4 session surveys and the Overall Conference survey (starting on Thursday) to receive your Cisco Live t-shirt.
- All surveys can be taken in the Cisco Events Mobile App or by logging in to the Content Catalog on ciscolive.com/emea.

Cisco Live sessions will be available for viewing on demand after the event at ciscolive.com.

Continue your education



Demos in the
Cisco Showcase



Walk-In Labs



Meet the Engineer
1:1 meetings



Related sessions



Thank you





You make **possible**